Introduction to the C Language

History, Fundamentals, and its Bridge to Hardware

Professor: Nerio Andrés Montoya
namontoy@unal.edu.co

2026-S2

Universidad Nacional de Colombia

Agenda

A Bit of History

Why C Matters

How C Works

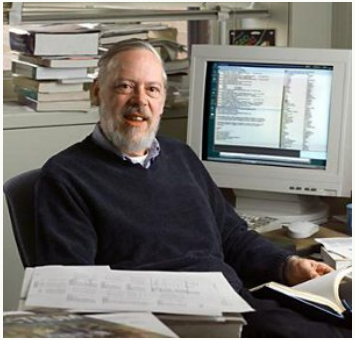
C Fundamentals

Closing

A Bit of History

The Creator

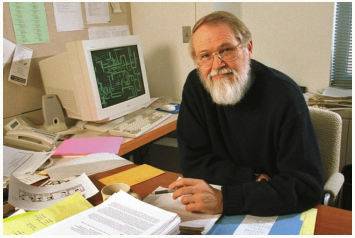
Dennis Ritchie (1941 – 2011)



- MSc. in Physics and Applied Mathematics
- Developed C in 1972 (Bell Labs)
- Co-creator of the Unix OS
- Co-author of the book “C”

The Writer

Brian Kernighan (1942 –)

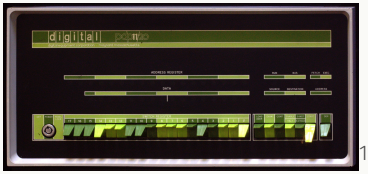


- Physics Engineer, PhD in Electronics
- Co-creator of the Unix OS
- Co-author of the book “C”

The Team

PDP-11 minicomputer





[1]

[1] Images taken from: *arstechnica.com*

Creators of UNIX and C

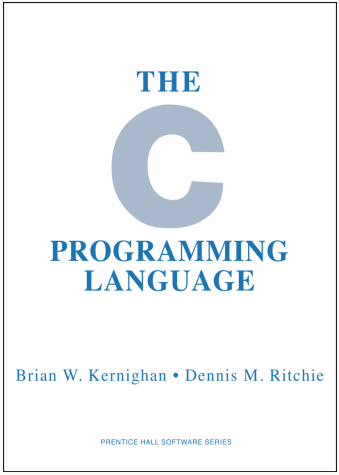
Dennis Ritchie and Ken Thompson, with a PDP-11/20.



[2]

[2] Image taken from: *arstechnica.com*

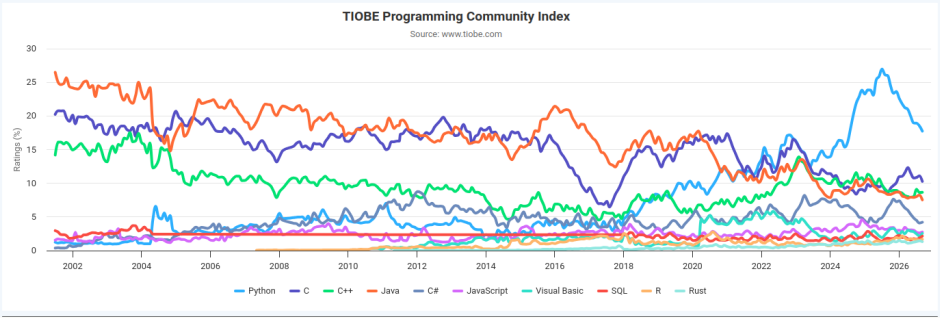
The “White Book” of C



Why C Matters

Why is C Important?

TIOBE programming community index



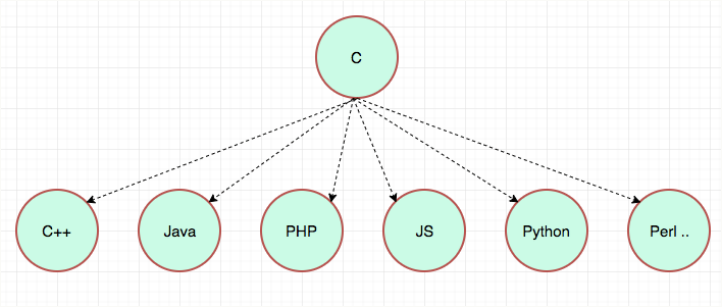
Why is C Important?

TIOBE programming community index

Sep 2026	Sep 2025	Change	Programming Language	Ratings	Change
1	**1**		**Python**	**17.76%**	**-8.22%**
2	3	↑	C	10.28%	+1.63%
3	2	↓	C++	8.67%	-0.13%
4	4		Java	7.54%	-0.81%
5	5		C#	4.22%	-2.16%
6	6		JavaScript	2.76%	-0.46%
7	7		Visual Basic	2.55%	-0.28%
8	11	↑	SQL	2.16%	+0.29%
9	13	⇈	R	1.69%	+0.27%
10	18	⇈	Rust	1.34%	+0.33%

Why is C Important?

It’s the father of all other languages...



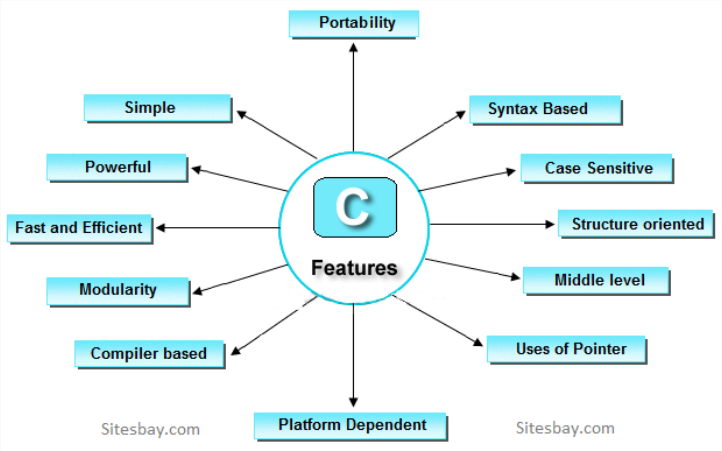
Why is C Important?

C was a major advancement; it was a language that was portable across CPUs and could generate efficient operating system code. C's success led to Objective C on the Mac, which led to today's Swift. Bell Labs took C and created C++. Sun Microsystems took C++ and created Java. Microsoft took Java and created C# and wrote .NET with it. Other languages derived from C include JavaScript, TypeScript, Go, and Rust.

But most notably, ATT Unix led to BSD Unix, which led to MacOS and then iOS. (ATT Unix also led to Linux and GNU, which led to RedHat, Ubuntu, SUSE, Debian, Gentoo, Slackware, and other source distributions.)

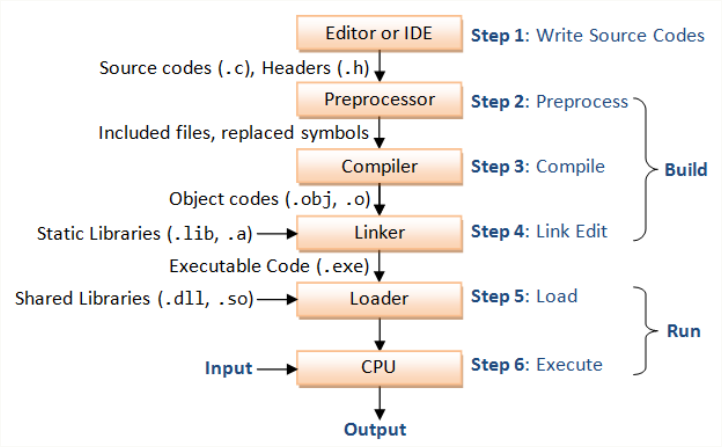
[2] Text taken from: *arstechnica.com*

Features of C

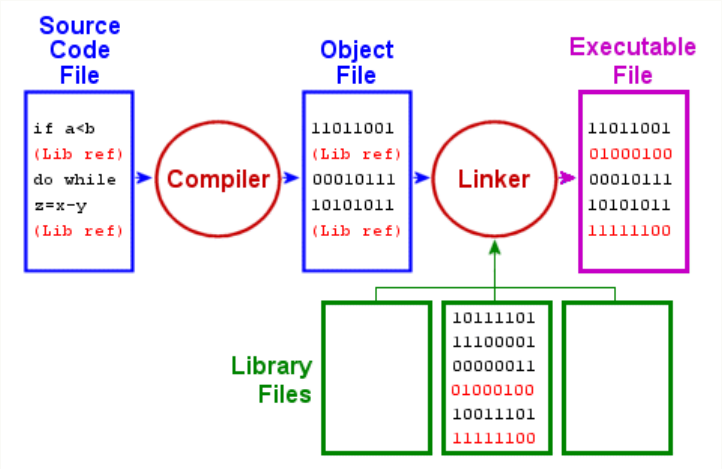


How C Works

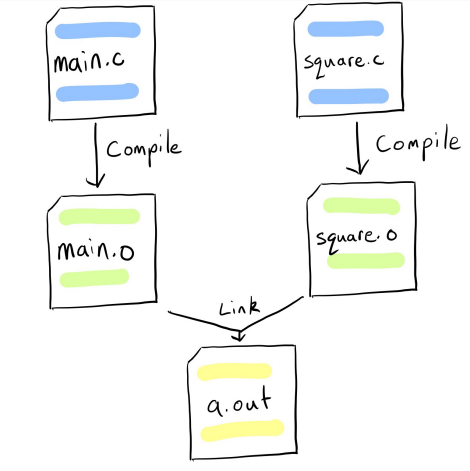
How Does C Work?



How Does C Work?



How Does C Work?



C Fundamentals

Quick Recap: Structure of a C Program

```c
#include <stdint.h>   // fixed-width integer types

int main(void)
{
    uint32_t count = 0;   // declaration + initialization

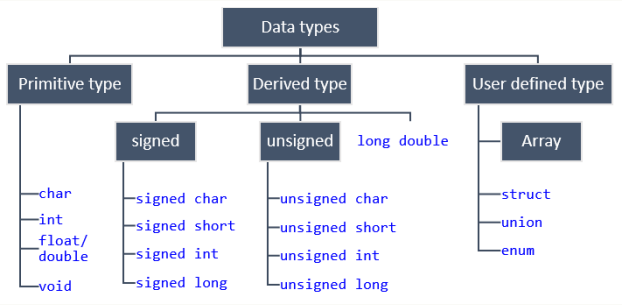
    count = count + 1;    // statement

    return 0;             // every path ends somewhere
}
```

- `main()` is the entry point — execution starts here
- Statements end in `;` comments: `//` or `/* */`

Basic Data Types

Further reading: *developer.arm.com*



Remember: Little-endian and the importance of size (operations, overflow, values, etc.)

[2] Image taken from: *codeforwin.org*

Data Types — and Why Sizes Matter Here

Type	Size	Range
`char`	1 byte	−128 to 127
`int`	*compiler-dependent!*	varies by platform
`uint8_t`	1 byte	0 to 255
`uint16_t`	2 bytes	0 to 65 535
`uint32_t`	4 bytes	0 to 4 294 967 295

- `int`'s size is **not guaranteed** across compilers or architectures
- Embedded code uses fixed-width types from `stdint.h` — a register is exactly 32 bits, always

C Keywords

32 Keywords in C Programming Language

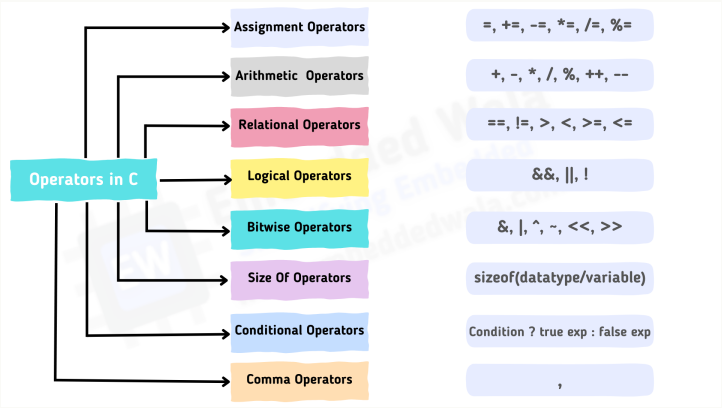
auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

The ARM C compiler also has an additional set of reserved keywords
developer.arm.com

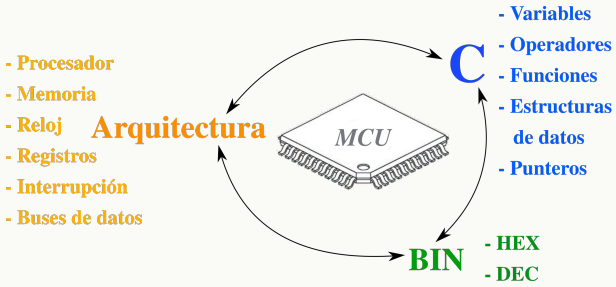
C Operators (A)

- arithmetic: `+`, `-`, `*`, `/`, `%`
- assignment: `=`
- augmented assignment: `+=`, `-=`, `*=`, `/=`, `%=`, `&=`, `|=`, `^=`, `<<=`, `>>=`
- bitwise logic: `~`, `&`, `|`, `^`
- bitwise shifts: `<<`, `>>`
- boolean logic: `!`, `&&`, `||`
- conditional evaluation: `? :`
- equality testing: `==`, `!=`
- calling functions: `( )`
- increment and decrement: `++`, `--`
- member selection: `.`, `->`
- object size: `sizeof`
- order relations: `<`, `<=`, `>`, `>=`
- reference and dereference: `&`, `*`, `[ ]`
- sequencing: `,`
- subexpression grouping: `( )`
- type conversion: `(typename)`

C Operators (B)



Looking Ahead: C → Hardware → BIN/HEX



- Next session: exactly how this compiles down to registers and individual bits on real hardware

Closing

Key Takeaways

- C has topped programming-language rankings for decades — and directly shaped nearly every language that followed
- Your C code compiles down to the exact instructions the Cortex-M4 executes — no hidden interpreter
- A program moves through preprocessing, compiling, and linking before it becomes a runnable binary
- Fixed-width types (`uint8_t`, `uint32_t`…) matter because registers have exact, fixed sizes
- Operators and keywords are the small, fixed vocabulary every C program is built from
- Next session: how these same ideas reach into real peripheral registers

printf("Hello, world\n");

